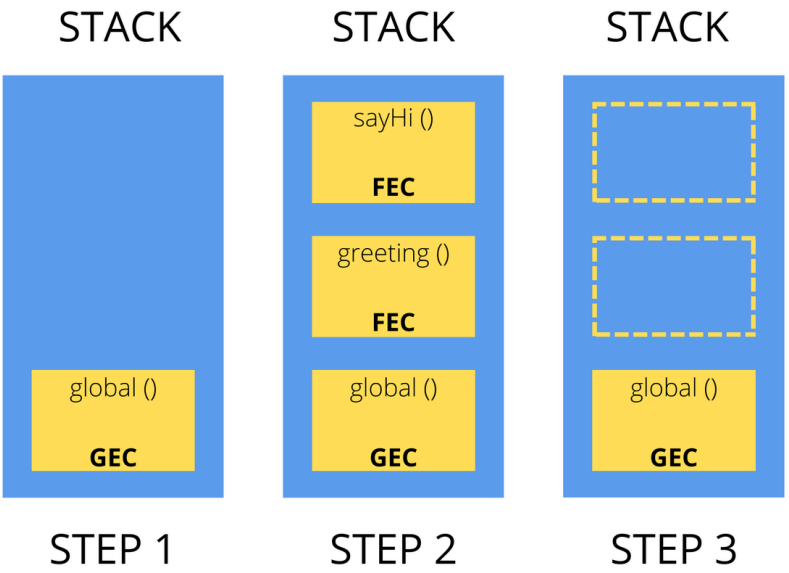
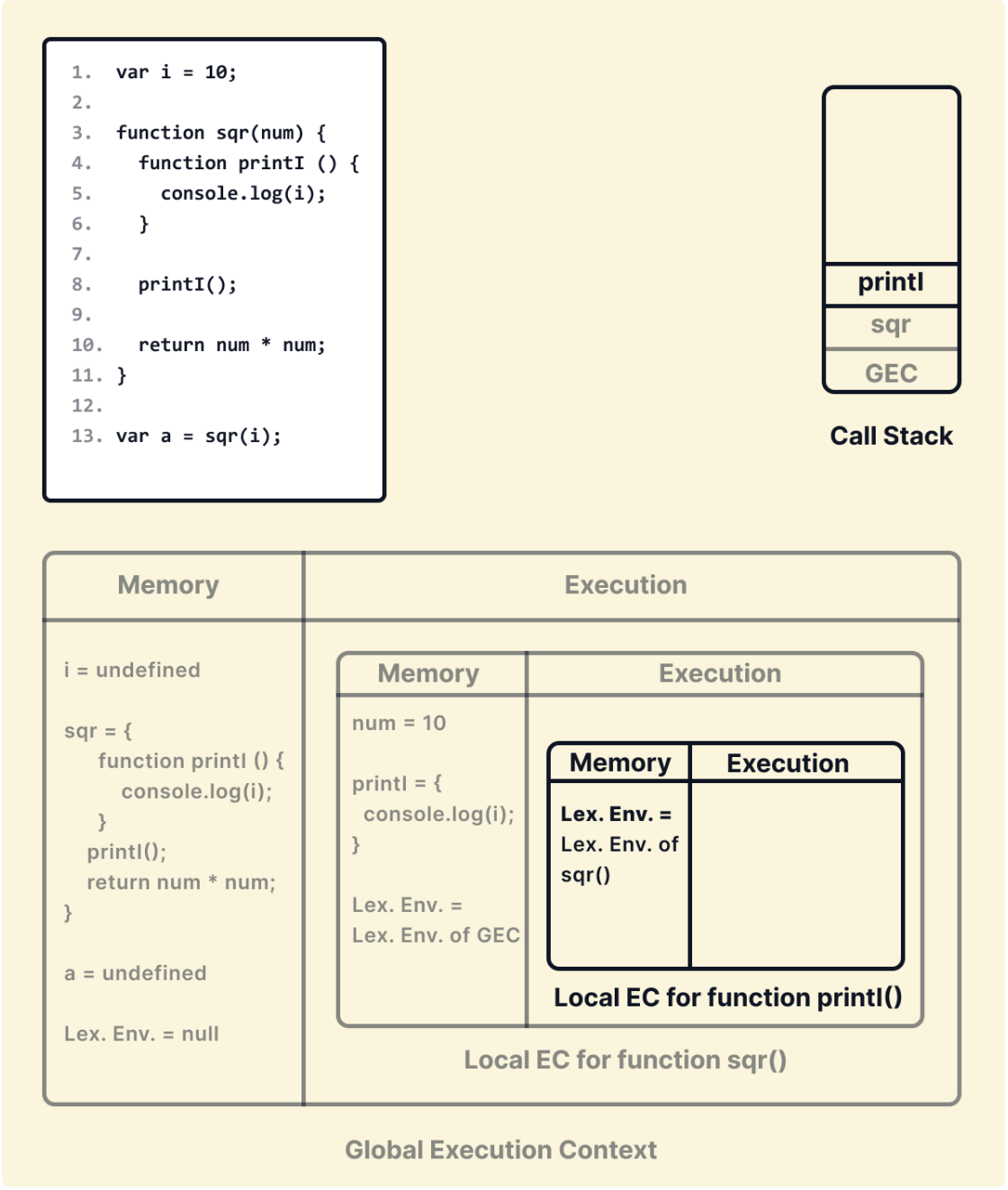
Global Execution Context (GEC)

- Can explain the **weird behavior of JS**





JavaScript Execution Context, Call Stack & Memory Management

1. Execution Context

An **Execution Context** is the environment in which JavaScript code is evaluated and executed.

JavaScript creates an execution context to manage variables, functions, and the execution of code.

There are three main types of execution contexts:

1. Global Execution Context (GEC)
2. Function Execution Context (FEC)
3. Eval Execution Context

Global Execution Context (GEC)

The **Global Execution Context** is the default execution environment created when a JavaScript program starts executing.

- It is created automatically.
- It manages global-level code, variables, and functions.
- It is pushed onto the Call Stack first.
- There is one GEC per JavaScript realm.

The GEC has two main phases:

1. Memory Creation Phase (aka. Variable Environment)

- JavaScript allocates memory for variables and functions.
- `var` variables are initialized with `undefined`.
- `let` and `const` are allocated but remain uninitialized.
- Function declarations are stored entirely in memory.

2. Code Execution Phase (aka. Thread of Execution)

- JavaScript executes the code line by line.
- Variables receive their assigned values.
- Functions execute when they are called.

2. Function Execution Context (FEC)

A **Function Execution Context** is created whenever a function is called.

Each function execution context manages its own:

- Variables and parameters.
- Local functions.
- Execution of function code.

Example:

```
function greet() {
    var message = "Hello JavaScript";
    console.log(message);
}

greet();
```

Execution flow:

1. GEC is created.

2. `greet()` is called.
3. A new FEC is created.
4. FEC is pushed onto the Call Stack.
5. The function executes inside its FEC.
6. After execution, FEC is popped from the Call Stack.
7. Execution returns to the calling context.

Important: A function declaration alone does not create an FEC. An FEC is created when the function is called.

3. Call Stack

The **Call Stack** is a mechanism JavaScript uses to manage execution contexts and their execution order.

It follows the LIFO principle:

LIFO = Last In, First Out

The most recently pushed execution context is removed first.

Example:

```
function first() {
    second();
}

function second() {
    console.log("Hello");
}

first();
```

Execution flow:

1. GEC is pushed onto the Call Stack.
2. `first()` is called and its FEC is pushed.
3. `second()` is called inside `first()` and its FEC is pushed.
4. `second()` finishes and its FEC is popped.
5. `first()` finishes and its FEC is popped.
6. Execution returns to GEC.

Call Stack while `second()` is executing:

```
        TOP
    +----------+
```

```
    | second() |
    +----------+
    | first()  |
    +----------+
    |   GEC    |
    +----------+
      BOTTOM
```

Remember:

- GEC is pushed first, so it stays at the bottom.
- Each function call creates a new FEC.
- A new FEC is pushed on top of the existing context.
- A completed FEC is popped from the stack.

4. Nested Function Execution

When a function calls another function, a new FEC is created on top of the existing FEC.

Example:

```
function first() {
    console.log("First function started");

    second();

    console.log("First function finished");
}

function second() {
    console.log("Second function executed");
}

first();
```

Execution order:

```
GEC created
    |
    v
first() called
    |
    v
FEC1 created
    |
    v
second() called
```

```
    |
    v
FEC2 created
    |
    v
second() executes
    |
    v
FEC2 popped
    |
    v
first() continues execution
    |
    v
FEC1 popped
    |
    v
Execution returns to GEC
```

The output is:

```
First function started
Second function executed
First function finished
```

5. Return Statement

The `return` statement sends a value from a function back to the calling context.

Example:

```
function add(a, b) {
    var result = a + b;
    return result;
}

var sum = add(10, 20);

console.log(sum);
```

Output:

```
30
```

Execution flow:

1. `add()` is called.

2. A new FEC is created.
3. Parameters receive values: `a = 10`, `b = 20`.
4. `result` becomes `30`.
5. `return result` sends `30` back to the calling context.
6. FEC is popped.
7. `sum` receives the returned value.

Important:

- `return` sends a value back to the caller.
- A function without an explicit return statement returns `undefined`.
- A function's FEC is removed after the function completes, whether it returns a value or not.

6. var, let and const in Memory Creation Phase

JavaScript handles `var`, `let`, and `const` differently during the Memory Creation Phase.

Example:

```
var a = 10;
let b = 20;
const c = 30;
```

During Memory Creation Phase:

Declaration	Initial state
`var a`	`undefined`
`let b`	Uninitialized
`const c`	Uninitialized

During Code Execution Phase:

Declaration	Value
`var a`	`10`
`let b`	`20`
`const c`	`30`

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between entering a scope and the point where a `let` or `const` declaration is initialized.

Accessing these variables before initialization causes a `ReferenceError`.

Example:

```
console.log(a); // undefined

// console.log(b); // ReferenceError
// console.log(c); // ReferenceError

var a = 10;
let b = 20;
const c = 30;
```

Differences between var, let and const

Feature	var	let	const
Scope	Function	Block	Block
Redeclaration	Allowed	Not allowed in same scope	Not allowed in same scope
Reassignment	Allowed	Allowed	Not allowed
Initial state	undefined	Uninitialized	Uninitialized
TDZ	No	Yes	Yes

Important: `const` prevents reassignment of the variable binding, but it does not make objects or arrays immutable.